Lesson 65: Depth-Width Tradeoffs

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Deep Neural Networks **3**

4 Depth Separation **4**
- 4.1 The Sawtooth Function . . . 4
- 4.2 The Depth Separation Theorem . . . 5

5 Width Separation and Width Sufficiency **7**
- 5.1 Width Sufficiency . . . 7
- 5.2 Linear Regions of ReLU Networks . . . 8

6 Approximation Rates for Smooth Functions **9**
- 6.1 Classical Approximation Rates . . . 9
- 6.2 Deep Network Rates for Sobolev Functions . . . 10
- 6.3 Compositional Functions and the Blessing of Depth . . . 10

7 Exercises **11**

8 Summary **12**

9 Further Reading **13**

1 Motivation and Overview

Lessons 63 and 64 established that single-hidden-layer networks are universal approximators, and that for functions in the Barron class the approximation rate is $O(1/m)$, independent of dimension. However, modern deep learning practice uses networks with many layers—not just one. Is depth merely an engineering convenience, or does it provide a fundamental mathematical advantage? Consider training a neural network to approximate a function that exhibits hierarchical or compositional structure, such as the mapping from pixels to object categories in a vision system. Empirically, deep networks vastly outperform shallow ones on such tasks, even when the total number of parameters is the same. This raises a precise question: *are there functions that deep networks with polynomial parameters can compute, but that shallow networks require exponentially many parameters to approximate?*

Telgarsky's 2016 depth separation theorem provides a striking affirmative answer: there exist explicit functions computable by $O(L)$-layer networks of constant width that cannot be approximated to constant accuracy by any network of depth $O(1)$ and width less than $2^{\Omega(L)}$. Complementing this, Lu et al. (2017) and Hanin (2019) show that *width* also matters: ReLU networks of width $d+4$ (for inputs in $\mathbb{R}^d$) are universal approximators regardless of depth. The tools developed in this lesson will let us understand precisely how depth and width contribute to the expressive power of neural networks, quantify the gains from depth for structured functions, and connect these results to classical approximation rates.

Why Depth-Width Tradeoffs Matter for RL/ML

- **Architecture design in deep RL.** Choosing the number of layers and width per layer in a DQN, PPO, or SAC network is a design decision. Depth separation results explain why deeper architectures can represent value functions with hierarchical structure more efficiently.
- **Compositional structure in value functions.** Many environments (e.g., Atari games, robotic manipulation) produce value functions with compositional structure $V^* = g_L \circ \cdots \circ g_1$. Deep networks can exploit this structure to avoid the curse of dimensionality, while shallow networks cannot.
- **Parameter efficiency.** In resource-constrained settings (embedded RL agents), understanding the depth-width tradeoff helps choose the most parameter-efficient architecture.
- **Linear regions and decision boundaries.** The number of linear regions in a ReLU network—exponential in depth but polynomial in width—determines the complexity of the decision boundaries the network can represent.

Prerequisites. Single-hidden-layer networks and the universal approximation theorem (Lesson 63, Theorems 4.1 and 5.2). Barron's approximation theorem and the Barron class (Lesson 64, Theorem 4.1). Piecewise-affine functions and the ReLU activation (Lesson 63, Example 5.5). Basic properties of Lipschitz functions and L^p norms (Lessons 2–4).

Learning objectives. After completing this lesson, you will be able to:

- **Define** deep neural networks with L layers and width W, the function class $\mathcal{F}(L, W, \sigma)$, and **compare** parameter counts of shallow vs. deep architectures.
- **State and prove** Telgarsky's depth separation theorem, including the sawtooth construction and the lower bound on the width required by shallow networks.
- **State** the width-sufficiency results of Lu et al. and Hanin, and **explain** the role of minimal width in universal approximation.

- **Derive** bounds on the number of linear regions of ReLU networks as a function of depth and width.
- **Apply** approximation rate results for smooth and compositionally structured functions to determine the advantage of deep over shallow architectures.
- **Justify** architecture choices in deep RL using depth-width tradeoff theory.

2 References

- Telgarsky [2016] — The depth separation theorem: benefits of depth via the sawtooth construction.
- Lu et al. [2017] — Width-$(d+4)$ ReLU networks are universal approximators; the expressive power from the width perspective.
- Hanin [2019] — Universal approximation with bounded width $d+1$ for L^p and width $d+1$ for $C(K)$ with one extra unit.
- Pinkus [1999] — Comprehensive survey of MLP approximation theory, including classical rate results.
- Montúfar et al. [2014] — Counting linear regions of deep ReLU networks; the combinatorial source of depth advantage.
- Yarotsky [2017] — Approximation rates for deep ReLU networks applied to Sobolev-class functions.

3 Deep Neural Networks

We formalize the class of functions computed by multi-layer (deep) neural networks, extending the single-hidden-layer definition from Lesson 63.

Definition 3.1 (L-Layer Neural Network)**.** Let $\sigma\colon \mathbb{R} \to \mathbb{R}$ be a fixed activation function. An *L-layer feedforward neural network* with layer widths $n_0, n_1, \ldots, n_L$ is a function $f\colon \mathbb{R}^{n_0} \to \mathbb{R}^{n_L}$ defined recursively by

$$\begin{aligned} h_0 &= x, \\ h_\ell &= \sigma(W_\ell h_{\ell-1} + b_\ell), \qquad \ell = 1, \ldots, L-1, \\ f(x) &= W_L h_{L-1} + b_L, \end{aligned} \tag{1}$$

where $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ is the weight matrix and $b_\ell \in \mathbb{R}^{n_\ell}$ is the bias vector for layer ℓ. The activation σ is applied componentwise. The integer L is called the *depth* and the last layer (without activation) is the *output layer*.

Definition 3.2 (Width and Depth)**.** Given an L-layer network with layer widths $n_0, n_1, \ldots, n_L$:

(i) The *depth* is L (the number of affine transformations).
(ii) The *width* is $W = \max_{1 \le \ell \le L-1} n_\ell$ (the maximum hidden-layer width).
(iii) The *total number of parameters* is $\sum_{\ell=1}^{L} (n_\ell \cdot n_{\ell-1} + n_\ell)$.

Definition 3.3 (Deep Network Function Class)**.** For depth L, width W, input dimension d, output dimension 1, and activation σ, define

$$\mathcal{F}(L, W, \sigma) = \big\{ f\colon \mathbb{R}^d \to \mathbb{R} : f \text{ is an } L\text{-layer network with width } \le W \text{ and activation } \sigma \big\}.$$

When $\sigma = \mathrm{ReLU}$, we write $\mathcal{F}(L, W)$ for brevity.

Example 3.4 (Parameter Count: Shallow vs. Deep)**.** Consider input dimension $d = 100$ and scalar output.

- **Shallow network** (depth $L = 2$, width m): The single hidden layer has $m(d+1) = 101m$ parameters (weights plus biases), and the output layer has $m + 1$ parameters. Total: $m(d + 2) + 1 = 102m + 1$. For $m = 1000$: 102,001 parameters.
- **Deep network** (depth $L = 10$, width $W = 100$): The first hidden layer has $100(100+1) = 10{,}100$ parameters. Each of the 8 intermediate hidden layers has $100(100 + 1) = 10{,}100$ parameters. The output layer has $100 + 1 = 101$ parameters. Total: $10{,}100 + 8 \times 10{,}100 + 101 = 91{,}001$ parameters.

Both architectures have roughly 10^5 parameters, but as we shall see, the deep network can represent exponentially more complex functions.

Remark 3.5. A single-hidden-layer network with m neurons is the special case $L = 2$, $n_0 = d$, $n_1 = m$, $n_2 = 1$. The function class $\mathcal{F}_m^\sigma$ from Lesson 63 coincides with $\mathcal{F}(2, m, \sigma)$. All results of Lessons 63–64 concern $L = 2$.

The central question is: for a fixed parameter budget, does distributing parameters across many layers (large L, moderate W) offer any advantage over concentrating them in a single layer (small L, large W)? The next section provides a dramatic affirmative answer.

4 Depth Separation

The depth separation theorem of Telgarsky [2016] shows that depth provides an exponential advantage in expressiveness. The key construction uses iterated piecewise-linear maps—the "sawtooth" function—that oscillate with a frequency that grows exponentially with depth.

4.1 The Sawtooth Function

Definition 4.1 (Tent Map)**.** The *tent map* $t\colon \mathbb{R} \to \mathbb{R}$ is defined by

$$t(x) = \begin{cases} 2x & \text{if } 0 \le x \le 1/2, \\ 2(1-x) & \text{if } 1/2 < x \le 1, \\ 0 & \text{otherwise,} \end{cases}$$

or equivalently $t(x) = 2\min(x, 1 - x)$ for $x \in [0, 1]$.

Lemma 4.2 (ReLU Representation of the Tent Map)**.** *The tent map t can be computed by a ReLU network of depth 2 and width 2. Specifically,*

$$t(x) = 2\max(0, x) - 4\max(0, x - 1/2) + 2\max(0, x - 1).$$

This uses 3 ReLU units, but can be reduced to 2 hidden units when restricted to $[0, 1]$ (since the third term vanishes).

Proof. For $x \in [0, 1/2]$: $\max(0, x) = x$, $\max(0, x-1/2) = 0$, $\max(0, x-1) = 0$, so the expression equals $2x = t(x)$. For $x \in (1/2, 1]$: $\max(0, x) = x$, $\max(0, x-1/2) = x-1/2$, $\max(0, x-1) = 0$, so the expression equals $2x - 4(x - 1/2) = 2x - 4x + 2 = 2(1 - x) = t(x)$. □

Definition 4.3 (Sawtooth Function)**.** For $L \ge 1$, the *L-fold sawtooth function* $s_L\colon [0, 1] \to [0, 1]$ is the L-fold composition of the tent map:

$$s_L = \underbrace{t \circ t \circ \cdots \circ t}_{L \text{ times}}.$$

We set $s_0(x) = x$ (the identity).

Lemma 4.4 (Properties of the Sawtooth). *The function s_L has the following properties:*

(i) s_L *is continuous and piecewise linear on* $[0, 1]$.
(ii) s_L *maps* $[0, 1]$ *onto* $[0, 1]$.
(iii) s_L *has exactly* 2^L *linear pieces (intervals of monotonicity), each of length* 2^{-L}.
(iv) s_L *oscillates between* 0 *and* 1 *exactly* 2^{L-1} *times (it has* 2^{L-1} *local maxima and* 2^{L-1} *local minima on* $(0, 1)$*, counting endpoints).*
(v) s_L *has Lipschitz constant* 2^L.

Proof. We prove these by induction on L.

Base case ($L = 1$): $s_1 = t$ has $2 = 2^1$ linear pieces (on $[0, 1/2]$ and $[1/2, 1]$), maps $[0, 1]$ onto $[0, 1]$, has one local maximum (at $x = 1/2$), and has Lipschitz constant $2 = 2^1$.

Inductive step: Assume s_{L-1} has 2^{L-1} linear pieces, each of length $2^{-(L-1)}$. On each such piece, s_{L-1} is an affine function mapping the piece onto $[0, 1]$ (by property (ii)). Composing with t replaces each linear piece with two sub-pieces (the tent map folds each $[0, 1]$ traversal into two pieces), giving $2 \cdot 2^{L-1} = 2^L$ linear pieces. The length of each piece is halved to 2^{-L}. The Lipschitz constant of $t \circ s_{L-1}$ is $\mathrm{Lip}(t) \cdot \mathrm{Lip}(s_{L-1}) = 2 \cdot 2^{L-1} = 2^L$. The oscillation count follows from the folding: each application of t doubles the number of maxima. □

Example 4.5 (Sawtooth Functions for Small L).

- $s_1 = t$: a single tent on $[0, 1]$, 2 linear pieces.
- $s_2 = t \circ t$: a "W"-shaped function with 4 linear pieces, oscillating between 0 and 1 twice.
- $s_3 = t \circ t \circ t$: 8 linear pieces, 4 oscillations.
- s_{10}: 1024 linear pieces, 512 oscillations—a rapidly oscillating function that looks like a dense sawtooth wave.

Proposition 4.6 (Sawtooth as a Deep Network). *For any $L \geq 1$, the sawtooth function s_L belongs to $\mathcal{F}(L + 1, 2)$: it can be computed by a ReLU network of depth $L + 1$ and width 2.*

Proof. By Lemma 4.2, the tent map t is computed by a depth-2, width-2 ReLU network. Since $s_L = t \circ t \circ \cdots \circ t$ (L times), we compose L copies of this network. The composition of L depth-2 networks yields a network of depth $L + 1$ (collapsing adjacent linear layers) with width 2 at every hidden layer. The total number of parameters is $O(L)$. □

4.2 The Depth Separation Theorem

Lemma 4.7 (Linear Pieces of Shallow ReLU Networks). *Any function $f \in \mathcal{F}(2, m)$ (a single-hidden-layer ReLU network with m hidden units and one-dimensional input) is piecewise linear with at most $m + 1$ linear pieces.*

Proof. Each hidden unit computes $\max(0, w_j x + b_j)$, which is piecewise linear with exactly 2 pieces (and one breakpoint at $x = -b_j/w_j$ when $w_j \neq 0$). The output $f(x) = a_0 + \sum_{j=1}^{m} a_j \max(0, w_j x + b_j)$ is a sum of m piecewise-linear functions on $\mathbb{R}$, each with one breakpoint. The set of breakpoints has at most m elements, so f is piecewise linear with at most $m + 1$ pieces. □

Lemma 4.8 (Oscillation Lower Bound). *Let $f, g\colon [0, 1] \to \mathbb{R}$ be continuous functions. If f has k points where f crosses a threshold τ (i.e., f changes from $f < \tau$ to $f > \tau$ or vice versa), and $\|f - g\|_\infty \leq \varepsilon < (\max f - \min f)/4$, then g must cross the threshold τ at least $k - 2$ times (for ε sufficiently small relative to the oscillation amplitude).*

More precisely, if f achieves values 0 *and* 1 *on alternating subintervals and $\|f - g\|_{L^1} \leq \delta$ with $\delta < 1/4$, then g must have at least as many sign changes from below* $1/2$ *to above* $1/2$ *as f does, minus a correction depending on δ.*

Proof. Since f oscillates between 0 and 1 with $k/2$ full oscillations, there are $k/2$ disjoint subintervals I_j of $[0,1]$ on which $f \geq 3/4$ and $k/2$ disjoint subintervals J_j on which $f \leq 1/4$, with the I_j and J_j alternating. If $\|f-g\|_{L^1} < \delta$ with δ small enough, then on most of each I_j, g must also be close to 1 (by Markov's inequality applied interval by interval), and on most of each J_j, g must be close to 0. For g to transition between these values, it must cross $1/2$ at least once between each pair of consecutive intervals, giving at least $k-1$ crossings. A piecewise linear function with p pieces can cross any threshold at most $p-1$ times. Therefore $p-1 \geq k-1$, i.e., $p \geq k$. □

Theorem 4.9 (Telgarsky, 2016 — Depth Separation)**.** *For any $L \geq 2$, there exists a function $f_L\colon [0,1] \to [0,1]$ such that:*

(i) $f_L \in \mathcal{F}(L+1,2)$: f_L is computed by a ReLU network of depth $L+1$ and width 2 (with $O(L)$ total parameters).

(ii) For any $g \in \mathcal{F}(2,m)$ (a depth-2 network with m hidden units),

$$\|f_L - g\|_{L^1[0,1]} \geq \frac{1}{8} \qquad \textit{whenever } m \leq 2^{L/2} - 1.$$

That is, approximating f_L within constant L^1 error requires a shallow network of width at least $2^{\Omega(L)}$.

Proof. **Construction.** Set $f_L = s_L$, the L-fold sawtooth function (Definition 4.3). By Proposition 4.6, $f_L \in \mathcal{F}(L+1,2)$, establishing (i).

Lower bound. By Lemma 4.4, $f_L = s_L$ has 2^L linear pieces and oscillates between 0 and 1 with 2^{L-1} full oscillations. We claim that any good approximation g must also have many linear pieces.

Consider the 2^{L-1} subintervals on which s_L rises from 0 to 1 (the "up-ramps"). Each up-ramp has length 2^{-L} and maps $[a_j, a_j + 2^{-L}]$ linearly onto $[0,1]$. Similarly, the 2^{L-1} "down-ramps" each map an interval of length 2^{-L} linearly from 1 to 0.

Now suppose $g \in \mathcal{F}(2,m)$ satisfies $\|f_L - g\|_{L^1} < 1/8$. By Lemma 4.7, g is piecewise linear with at most $m+1$ pieces. We show this forces $m \geq 2^{L/2}$.

On each of the 2^{L-1} subintervals where $s_L \geq 3/4$ (the top halves of the up-ramps, totaling length $2^{L-1} \cdot 2^{-(L+1)} = 1/4$), if g is far from s_L on a fraction larger than $1/2$ of this region, the L^1 error exceeds $(1/4)(1/4)(1/2) = 1/32$ from these intervals alone. Similarly for the intervals where $s_L \leq 1/4$. For $\|f_L - g\|_{L^1} < 1/8$, the function g must be above $1/2$ on most of the intervals where s_L is near 1, and below $1/2$ on most of the intervals where s_L is near 0. Since these intervals alternate, g must cross the level $1/2$ at least $\Omega(2^{L-1})$ times.

Each linear piece of g can cross the level $1/2$ at most once. Therefore g must have at least $\Omega(2^{L-1})$ linear pieces, which means $m+1 \geq 2^{L-1}$, i.e., $m \geq 2^{L-1} - 1$. The sharper bound $m \geq 2^{L/2} - 1$ stated in the theorem follows from a more refined analysis of the L^1 error on individual pieces: a piecewise linear function with p pieces on $[0,1]$ can approximate a function with k oscillations of unit amplitude only if $p \geq \sqrt{k}$ when measured in L^1 (since the L^1 error per oscillation period that is not tracked by a linear piece is $\Omega(1/p)$, giving total error $\Omega(k/p)$; setting this less than $1/8$ requires $p \geq 8k$, but the $\sqrt{k}$ bound comes from a counting argument: each piece of g can "cancel" at most $O(p)$ of the oscillations, so $p^2 \geq k$ is needed). The exact constant and the exponent $L/2$ in the lower bound are established by Telgarsky [2016]. □

Telgarsky's Depth Separation

There exist functions computable by ReLU networks of depth $O(L)$ and width $O(1)$ that require width $2^{\Omega(L)}$ to be approximated (in L^1 norm) by any network of depth $O(1)$. Depth provides an **exponential** advantage in expressiveness over width.

Example 4.10 (Depth-2 vs. Depth-20)**.** The sawtooth s_{20} has $2^{20} \approx 10^6$ linear pieces and is computed by a depth-21 network with width 2 ($O(20)$ parameters). Approximating it within L^1 error $1/8$ with a single-hidden-layer network requires width at least $2^{10} = 1024$. The deep network uses roughly 60 parameters; the shallow network needs at least $2 \times 1024 + 1025 = 3073$ parameters. The ratio is roughly $50\times$.

Example 4.11 (Depth Separation in Higher Dimensions)**.** Telgarsky's construction is one-dimensional, but it extends to higher dimensions by applying the sawtooth along a coordinate projection. Eldan and Shamir [2016] proved a depth separation result in $\mathbb{R}^d$ with a different construction: there exists a radial function on $\mathbb{R}^d$ expressible by a depth-3 network of polynomial width, for which any depth-2 network requires width $\Omega(2^d)$ to approximate. This shows that depth separation is not merely a one-dimensional phenomenon.

Depth Separation and RL Architectures

In practice, deep RL agents use networks with 3–10 hidden layers, not single-hidden-layer networks. The depth separation theorem provides theoretical justification: if the value function V^* or the policy π^* has structure resembling iterated folding (which is plausible for environments with hierarchical dynamics), then deep networks can represent it with polynomially fewer parameters than shallow ones. This is consistent with the empirical observation that deeper networks improve performance in complex environments (e.g., Atari, MuJoCo) even when the total parameter count is controlled.

The depth separation theorem shows that depth provides an exponential advantage. We now ask the complementary question: is *width* also important?

5 Width Separation and Width Sufficiency

While Telgarsky's theorem shows that depth can be exponentially more efficient than width, the following results of Lu et al. and Hanin show that sufficient width also guarantees universal approximation, even at fixed depth.

5.1 Width Sufficiency

Theorem 5.1 (Lu et al., 2017 — Width Sufficiency for ReLU Networks)**.** *Let $d \geq 1$. The class of ReLU networks with input dimension d and width $w \geq d + 4$ is a universal approximator of $L^1(\mathbb{R}^d, \mu)$ for any compactly supported probability measure μ on $\mathbb{R}^d$: for any $f \in L^1(\mu)$ and $\varepsilon > 0$, there exists a ReLU network of width w (and some finite depth L) such that*

$$\int_{\mathbb{R}^d} |f(x) - g(x)| \, d\mu(x) < \varepsilon.$$

Proof sketch. The proof of Lu et al. [2017] proceeds by showing that width-$(d+4)$ networks can implement certain "building block" operations:

1. **Step functions:** A width-$(d + 4)$ ReLU network of sufficient depth can approximate indicator functions of hyperrectangles to arbitrary accuracy.
2. **Lebesgue integral:** Since simple functions (finite linear combinations of indicator functions of measurable sets) are dense in $L^1(\mu)$, it suffices to approximate these. Hyperrectangle indicators generate the Borel σ-algebra, so step (1) suffices.
3. **Propagation:** The key technical argument shows that a ReLU network of width $d + 4$ can (a) compute the identity map on $\mathbb{R}^d$ (using d units), (b) use 2 additional units to implement a tent-map fold, and (c) use 2 more units for bookkeeping. By composing many layers, the network can construct arbitrarily fine-grained step functions.

The full proof requires careful analysis of how ReLU units propagate information through layers; we refer to Lu et al. [2017] for details. □

Theorem 5.2 (Hanin, 2019 — Minimal Width)**.** *Let $K \subseteq \mathbb{R}^d$ be compact and let μ be a probability measure on K with full support.*

(i) For $1 \le p < \infty$: the class of ReLU networks of width $w \ge d+1$ is dense in $L^p(K, \mu)$.

(ii) For uniform approximation: the class of ReLU networks of width $w \ge d+1$ is dense in $C(K)$.

(iii) The bound $w \ge d+1$ is tight: width-d ReLU networks are not *dense in $L^p(K, \mu)$ for any $p \ge 1$.*

Proof sketch. (iii) Width-d ReLU networks compute functions $f\colon \mathbb{R}^d \to \mathbb{R}$ whose level sets $\{x : f(x) = c\}$ have a specific geometric structure imposed by the bottleneck layer of width d. Specifically, at any hidden layer of width exactly d, the network factors through a piecewise-linear map $\mathbb{R}^d \to \mathbb{R}^d$, which has determinant zero on a measure-zero set and is injective almost everywhere. This prevents the network from "folding" the input space enough to approximate functions with certain topological properties. The detailed argument uses the piecewise-linear structure and degree theory; see Hanin [2019].

(i)–(ii) The positive results follow by showing that width-$(d+1)$ networks can implement the identity map $\mathbb{R}^d \to \mathbb{R}^d$ while retaining one extra unit for folding. The folding operation, iterated over many layers, generates arbitrarily complex piecewise-linear functions, analogous to the sawtooth construction in Section 4 but in higher dimensions. □

Example 5.3 (Minimal Width in One Dimension)**.** For $d = 1$, the minimal width for universal approximation is $d + 1 = 2$. Width-1 networks compute functions of the form $f(x) = a_L \max(0, a_{L-1} \max(0, \cdots) + b_{L-1}) + b_L$, which are piecewise linear with at most 2 pieces regardless of depth (each ReLU applied to a monotone function can add at most one breakpoint, but with width 1, only one breakpoint is produced at each layer, and subsequent layers may remove breakpoints). Therefore width-1 networks are not universal, confirming $w \ge 2 = d + 1$.

5.2 Linear Regions of ReLU Networks

The depth-versus-width advantage for ReLU networks can be understood through the lens of *linear regions*: the number of connected regions of the input space on which the network computes a single affine function.

Definition 5.4 (Linear Region)**.** A *linear region* of a piecewise-affine function $f\colon \mathbb{R}^d \to \mathbb{R}$ is a maximal connected subset of $\mathbb{R}^d$ on which f is affine.

Theorem 5.5 (Linear Region Bounds)**.** *Let $f \in \mathcal{F}(L, W)$ be a ReLU network with depth L and width W on $\mathbb{R}^d$.*

(i) ***Upper bound:*** *f has at most $\binom{W}{d}^{L-1} \cdot 2^W$ linear regions. When $W \ge d$, this simplifies to at most $\left(\frac{eW}{d}\right)^{d(L-1)} \cdot 2^W$. In particular, the number of linear regions is $O(W^{d(L-1)})$ — exponential in L and polynomial in W (for fixed d).*

(ii) ***Lower bound:*** *There exist ReLU networks of depth L and width W with at least $\lfloor W/d \rfloor^{d(L-1)}$ linear regions.*

Proof sketch. (i) Each hidden layer has W ReLU units. Each unit partitions $\mathbb{R}^d$ into two half-spaces (the unit's active and inactive regions). The arrangement of W hyperplanes in $\mathbb{R}^d$ creates at most $\sum_{i=0}^{d} \binom{W}{i} \le \binom{W}{d} + \binom{W}{d-1} + \cdots$ regions (by Zaslavsky's theorem for hyperplane arrangements). When $W \ge d$, this is at most $\left(\frac{eW}{d}\right)^d$. Each layer can multiply the number of linear regions by this factor, so $L - 1$ hidden layers give at most $\left(\frac{eW}{d}\right)^{d(L-1)}$ regions. The last hidden layer contributes at most 2^W (with a looser bound that is tighter when W is small).

(ii) The lower bound is achieved by networks where each layer folds the input space maximally. The construction in Montúfar et al. [2014] uses each group of d units to implement a fold (doubling the number of regions), and W/d such groups per layer give $\lfloor W/d \rfloor^d$ new regions per layer. □

Example 5.6 (Linear Regions: Depth vs. Width)**.** For $d = 10$, a network with $W = 100$ and $L = 2$ (shallow) has at most $\binom{100}{10} \approx 1.7 \times 10^{13}$ linear regions. A network with $W = 20$ and $L = 10$ (deep) can have up to $\binom{20}{10}^9 \approx (184{,}756)^9 \approx 10^{47}$ linear regions. The deep network has far fewer parameters (≈ 4000) than the shallow one ($\approx 11{,}000$) but can create exponentially more linear regions.

Depth vs. Width in Linear Region Count

For ReLU networks with input dimension d, the maximum number of linear regions is:

- **Exponential in depth**: $\Theta(W^d)^{L-1}$ regions for depth-L, width-W networks.
- **Polynomial in width**: $O(W^d)$ regions for single-hidden-layer (depth-2) networks with W units.

Depth provides exponentially more linear regions per parameter than width.

Linear Regions Do Not Equal Approximation Quality

A large number of linear regions is *necessary* for approximating highly oscillatory functions but is not *sufficient* for good approximation. The regions must be arranged to match the target function. The optimization landscape (how gradient descent navigates the parameter space to find the right arrangement) is a separate and harder question, addressed by the neural tangent kernel theory in Lesson 66.

We now quantify the approximation rates that deep networks achieve for specific function classes.

6 Approximation Rates for Smooth Functions

The depth separation results of Section 4 use pathological functions (sawtooths) to prove worst-case separation. In this section, we study approximation rates for natural function classes—Sobolev spaces and compositionally structured functions—and show that deep networks achieve provably better rates than shallow ones.

6.1 Classical Approximation Rates

We recall the classical approximation rate for functions in Sobolev spaces, previously stated in Lesson 64 (Proposition 5.1).

Definition 6.1 (Sobolev Space)**.** For $s \geq 1$ and $1 \leq p \leq \infty$, the *Sobolev space* $\mathcal{W}^{s,p}([0,1]^d)$ consists of all functions $f\colon [0,1]^d \to \mathbb{R}$ whose partial derivatives up to order s exist in the weak sense and belong to L^p, equipped with the norm

$$\|f\|_{\mathcal{W}^{s,p}} = \sum_{|\alpha| \leq s} \|D^\alpha f\|_{L^p}.$$

The Sobolev ball of radius B is $\mathcal{W}^{s,p}_B = \{f : \|f\|_{\mathcal{W}^{s,p}} \leq B\}$.

Proposition 6.2 (Classical Minimax Rates)**.** *For any nonlinear approximation scheme using m parameters (including neural networks, splines, wavelets), the minimax L^2 approximation rate over $\mathcal{W}_B^{s,2}([0,1]^d)$ satisfies*

$$\inf_{\hat{f}_m} \sup_{f \in \mathcal{W}_B^{s,2}} \|f - \hat{f}_m\|_{L^2} = \Theta(B \cdot m^{-s/d}).$$

The rate $m^{-s/d}$ suffers the curse of dimensionality*: for fixed smoothness s and large d, the rate becomes arbitrarily slow.*

Proof. The lower bound follows from Kolmogorov n-widths or metric entropy arguments; see Pinkus [1999]. The upper bound is achieved by tensor-product B-splines or wavelet expansions with m terms. □

6.2 Deep Network Rates for Sobolev Functions

Deep ReLU networks can match the classical minimax rate with an explicit construction.

Theorem 6.3 (Yarotsky, 2017)**.** *For $f \in \mathcal{W}^{s,\infty}([0,1]^d)$ with $\|f\|_{\mathcal{W}^{s,\infty}} \le B$, and for any $\varepsilon > 0$, there exists a ReLU network g with depth $O(\log(1/\varepsilon))$ and total parameters $O(\varepsilon^{-d/s}\log(1/\varepsilon))$ such that $\|f - g\|_{L^\infty} \le \varepsilon$.*

The approximation rate as a function of the number of parameters m is

$$\|f - g\|_{L^\infty} = O\big(m^{-s/d} \cdot (\log m)^{s/d}\big),$$

which matches the classical rate $m^{-s/d}$ up to logarithmic factors.

Proof sketch. The proof of Yarotsky [2017] proceeds in three steps:

Step 1: Approximate monomials. Using the identity $xy = \frac{1}{2}[(x+y)^2 - x^2 - y^2]$ and the fact that x^2 can be approximated by a deep ReLU network of depth $O(\log(1/\varepsilon))$ with error ε (via a binary expansion construction), one shows that products $x_i x_j$ can be approximated. By iteration, arbitrary monomials $x_1^{k_1} \cdots x_d^{k_d}$ with $\sum k_j \le s$ can be approximated.

Step 2: Local Taylor approximation. Partition $[0,1]^d$ into N^d cubes of side length $1/N$. On each cube, approximate f by its Taylor polynomial of degree $s-1$. By Taylor's theorem, the local error is $O(N^{-s})$.

Step 3: Assemble. Using the ReLU network to implement the partition of unity and the local polynomial approximations, the total number of parameters is $O(N^d \cdot \mathrm{poly}(\log N))$. Setting $N = \varepsilon^{-1/s}$ gives total parameters $m = O(\varepsilon^{-d/s}\log(1/\varepsilon))$, which achieves $\|f - g\|_{L^\infty} \le C\varepsilon$. □

Remark 6.4. Theorem 6.3 uses depth $O(\log(1/\varepsilon))$ to approximate monomials via the squaring trick. A shallow network would need width $O(\varepsilon^{-d/s})$ without the logarithmic depth factor. The depth is essential for efficiently computing multiplications, which are the building blocks of polynomial approximation.

6.3 Compositional Functions and the Blessing of Depth

The most dramatic advantage of depth appears for functions with *compositional structure*: functions that decompose as $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$ where each f_ℓ operates on a low-dimensional input.

Definition 6.5 (Compositional Function)**.** A function $f\colon \mathbb{R}^d \to \mathbb{R}$ has (L, d_0)-*compositional structure* if there exist functions $g_1, g_2, \ldots, g_L$ with $g_\ell\colon \mathbb{R}^{d_0} \to \mathbb{R}^{d_0}$ (for $\ell < L$) and $g_L\colon \mathbb{R}^{d_0} \to \mathbb{R}$, each with s bounded derivatives, such that

$$f = g_L \circ g_{L-1} \circ \cdots \circ g_1 \circ P,$$

where $P\colon \mathbb{R}^d \to \mathbb{R}^{d_0}$ is a fixed projection (or partition of coordinates into groups of size d_0).

Theorem 6.6 (Depth Advantage for Compositional Functions)**.** *Let f be an (L, d_0)-compositional function where each component $g_\ell \in \mathcal{W}^{s,\infty}(\mathbb{R}^{d_0})$ with bounded norm. Then:*

*(i) A **deep network** matching the compositional structure (depth $O(L\log(1/\varepsilon))$, width $O(d_0)$) achieves*

$$\|f - g\|_{L^\infty} \le \varepsilon \quad \text{with } O\big(L \cdot \varepsilon^{-d_0/s}\log(1/\varepsilon)\big) \text{ parameters.}$$

The dependence on the intrinsic dimension d_0 replaces the ambient dimension d.

*(ii) A **shallow network** (depth 2), unaware of the compositional structure, requires $\Omega(\varepsilon^{-d/s})$ parameters to achieve the same accuracy, suffering the curse of dimensionality in d.*

Proof sketch. (i) Apply Theorem 6.3 to each component g_ℓ separately, using a sub-network of depth $O(\log(1/\varepsilon))$ and $O(\varepsilon^{-d_0/s}\log(1/\varepsilon))$ parameters. Compose the L sub-networks. The errors accumulate multiplicatively (due to the Lipschitz constants of the components), but since each g_ℓ has bounded Lipschitz constant, the total error is $O(L\varepsilon)$ (rescaling ε absorbs the factor L). The total parameter count is L times the per-component cost.

(ii) A shallow network treats f as a function on $\mathbb{R}^d$ without exploiting the compositional structure, so the classical rate $\Theta(\varepsilon^{-d/s})$ applies. □

Example 6.7 (Vision-Like Compositional Structure)**.** Consider classifying images of size 32×32 pixels ($d = 1024$). Suppose the classification function has a 3-level hierarchy: (1) local edge detectors operate on 5×5 patches ($d_0 = 25$), (2) mid-level feature combiners aggregate edge outputs, and (3) a global classifier combines mid-level features. Each level has smoothness $s = 2$.

- **Shallow network:** Rate $\varepsilon^{-d/s} = \varepsilon^{-512}$. For $\varepsilon = 0.01$: 10^{1024} parameters.
- **Deep network:** Rate $L \cdot \varepsilon^{-d_0/s} = 3 \cdot \varepsilon^{-12.5}$. For $\varepsilon = 0.01$: 3×10^{25} parameters.

The deep network reduces the exponent from 512 to 12.5—a dramatic improvement.

Hierarchical Structure in RL

Many RL environments exhibit compositional structure:

- **Atari games**: The screen is processed hierarchically (pixels → edges → objects → game state).
- **Robotic control**: Joint angles → limb positions → body configuration → task-relevant features.
- **Multi-agent systems**: Individual agent states → local interactions → group behavior → global outcome.

In each case, the value function inherits the compositional structure of the environment, and deep networks can exploit this structure via Theorem 6.6, while shallow networks cannot.

Remark 6.8. The results of this section contextualize Pinkus [1999], who surveyed the approximation theory of the MLP model before the deep learning revolution. Pinkus focused on single-hidden-layer networks and established the minimax rate $\Theta(m^{-s/d})$ for Sobolev functions. The key insight of the subsequent work by Telgarsky, Yarotsky, and others is that *depth* allows networks to exploit structure (compositionality, multiplications, recursive folding) in ways that are impossible for shallow networks.

7 Exercises

Exercise 7.1 (Sawtooth Function Computation)**.** (*)
Compute $s_3(x)$ explicitly for $x \in [0, 1]$. List all breakpoints, verify that there are $2^3 = 8$ linear pieces, and sketch the graph.

Exercise 7.2 (Linear Regions of a Specific Network)**.** (*)
Consider a ReLU network on $\mathbb{R}^2$ with depth $L = 3$ and width $W = 4$.

(a) Compute the upper bound on the number of linear regions from Theorem 5.5.

(b) Construct a specific network achieving at least 16 linear regions.

Exercise 7.3 (Depth Separation for L^2 Approximation)**.** (**)
Show that $\|s_L\|^2_{L^2[0,1]} = 1/3$ for all $L \geq 1$. Using this, strengthen the depth separation bound of Theorem 4.9 to the L^2 norm: show that any $g \in \mathcal{F}(2, m)$ with $m < 2^{L/2}$ satisfies $\|s_L - g\|_{L^2} \geq c$ for a constant $c > 0$.

Hint: Use the orthogonality of s_L on its linear pieces and the fact that a piecewise linear function with $m + 1$ pieces has $\|g\|^2_{L^2}$ that can be computed explicitly.

Exercise 7.4 (Approximating Products with Deep ReLU Networks)**.** (**)
The key building block in Yarotsky's construction (Theorem 6.3) is approximating x^2 on $[0, 1]$.

(a) Show that the sawtooth function s_L satisfies $\|x - x^2 - \frac{1}{4}s_L(x)\|_{L^\infty[0,1]} = O(4^{-L})$ by considering the map $g(x) = 4x(1 - x)$ and its iterates. (Actually, use the identity $x^2 = x - \sum_{k=1}^{L} 2^{-2k} s_k(x) + O(4^{-L})$.)

(b) Conclude that x^2 can be approximated to accuracy ε by a ReLU network of depth $O(\log(1/\varepsilon))$ and width $O(1)$.

(c) Using $xy = \frac{1}{2}[(x+y)^2 - x^2 - y^2]$, show that multiplication can be approximated by a ReLU network of depth $O(\log(1/\varepsilon))$ and width $O(1)$.

Exercise 7.5 (Width-d Networks Are Not Universal)**.** (**)
Let $d = 2$ and consider a ReLU network of width 2 and arbitrary depth L.

(a) Show that each hidden layer computes a piecewise-affine map $\mathbb{R}^2 \to \mathbb{R}^2$. Argue that the composition of L such maps is also piecewise affine.

(b) Show that the resulting function $f\colon \mathbb{R}^2 \to \mathbb{R}$ has level sets $\{x : f(x) = c\}$ that are piecewise linear.

(c) Conclude that f cannot approximate the function $g(x_1, x_2) = x_1^2 + x_2^2$ uniformly on $[-1, 1]^2$, because g has curved level sets. This confirms that width $d = 2$ is insufficient for universal approximation in $C([-1, 1]^2)$.

Exercise 7.6 (Deep RL Architecture Design)**.** (***)
Consider an RL environment where the state is a vector $s = (s_1, \ldots, s_{100}) \in \mathbb{R}^{100}$ and the optimal value function has the compositional structure $V^*(s) = h(g_1(s_1, \ldots, s_{10}), g_2(s_{11}, \ldots, s_{20}), \ldots, g_{10}(s_{91}, \ldots, s_{100}))$, where each $g_k\colon \mathbb{R}^{10} \to \mathbb{R}$ and $h\colon \mathbb{R}^{10} \to \mathbb{R}$ are twice continuously differentiable ($s = 2$).

(a) Apply Theorem 6.6 to determine the parameter count needed by a deep network to achieve ε-approximation.

(b) Compute the parameter count for a shallow network.

(c) Design a specific network architecture (layer widths and depth) that exploits the compositional structure. What is the total parameter count?

8 Summary

- An L-layer neural network applies L affine transformations interleaved with activations; the **depth** L and **width** $W = \max_\ell n_\ell$ are the key architectural parameters.
- The **sawtooth function** $s_L = t^{\circ L}$ (iterated tent maps) has 2^L linear pieces and can be computed by a depth-$(L + 1)$, width-2 ReLU network with $O(L)$ parameters.

- **Telgarsky's depth separation theorem**: the sawtooth s_L requires width $2^{\Omega(L)}$ to be approximated by a depth-2 network. Depth provides an exponential advantage in expressiveness.
- **Width sufficiency**: ReLU networks of width $d+4$ (Lu et al.) or $d+1$ (Hanin) are universal approximators regardless of the target function, provided depth is allowed to grow. Width d is provably insufficient.
- The number of **linear regions** of a ReLU network grows exponentially in depth but polynomially in width, providing a combinatorial explanation for depth's advantage.
- For **Sobolev-class functions**, deep ReLU networks achieve the classical minimax rate $O(m^{-s/d})$ using depth $O(\log(1/\varepsilon))$, which is needed to efficiently compute polynomial building blocks via the squaring trick.
- For **compositionally structured functions** $f = g_L \circ \cdots \circ g_1$, deep networks achieve approximation rates depending on the intrinsic dimension d_0 of each component, not the ambient dimension d—avoiding the curse of dimensionality entirely.
- The depth-width tradeoff justifies the use of deep architectures in RL/ML: environments with hierarchical structure produce value functions that deep networks can approximate exponentially more efficiently than shallow networks.

9 Further Reading

- **Lesson 66** develops the neural tangent kernel framework, which addresses the optimization question: can gradient descent find good weights in the deep networks whose existence this lesson establishes? It also provides Rademacher complexity bounds for deep networks that complement the approximation results here.
- **Lesson 67** (RNNs and BPTT) studies recurrent architectures, where "depth" takes the form of unrolling through time. The depth separation results of this lesson apply (with modifications) to unrolled RNNs.
- Eldan and Shamir [2016] proves a depth separation result in $\mathbb{R}^d$ showing that three-layer networks can represent functions that two-layer networks require exponential width to approximate. The construction uses a different technique (random feature functions) than Telgarsky's sawtooth.
- Safran and Shamir [2017] extends depth-width tradeoffs to "natural" functions (not just adversarial constructions), showing that radial functions benefit from depth.
- Poggio et al. [2017] provides a detailed analysis of when and why depth avoids the curse of dimensionality, focusing on compositional functions arising in vision and language.
- Schmidt-Hieber [2020] establishes minimax-optimal rates for deep ReLU networks in nonparametric regression, showing that deep networks adapt to low-dimensional structure in the regression function.
- **Historical note.** The question of depth versus width in neural networks dates back to the 1990s (e.g., the work of Håstad on Boolean circuit complexity inspired analogous questions for neural networks). The modern depth separation results of Telgarsky (2016) and Eldan–Shamir (2016) provided the first unconditional separations for standard neural network architectures, settling a longstanding open question.

References

Ronen Eldan and Ohad Shamir. The power of depth for feedforward neural networks. *Conference on Learning Theory*, pages 907–940, 2016.

Boris Hanin. Universal function approximation by deep neural nets with bounded width and ReLU activations. *Mathematics*, 7(10):992, 2019.

Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, and Liwei Wang. The expressive power of neural networks: A view from the width. *Advances in Neural Information Processing Systems*, 30:6231–6239, 2017.

Guido F. Montúfar, Razvan Pascanu, Kyunghyun Cho, and Yoshua Bengio. On the number of linear regions of deep neural networks. *Advances in Neural Information Processing Systems*, 27:2924–2932, 2014.

Allan Pinkus. Approximation theory of the MLP model in neural networks. *Acta Numerica*, 8: 143–195, 1999.

Tomaso Poggio, Hrushikesh Mhaskar, Lorenzo Rosasco, Brando Miranda, and Qianli Liao. Why and when can deep—but not shallow—networks avoid the curse of dimensionality: A review. *International Journal of Automation and Computing*, 14(5):503–519, 2017.

Itay Safran and Ohad Shamir. Depth-width tradeoffs in approximating natural functions with neural networks. *Proceedings of the 34th International Conference on Machine Learning*, pages 2979–2987, 2017.

Johannes Schmidt-Hieber. Nonparametric regression using deep neural networks with ReLU activation function. *The Annals of Statistics*, 48(4):1875–1897, 2020.

Matus Telgarsky. Benefits of depth in neural networks. *Conference on Learning Theory*, pages 1517–1539, 2016.

Dmitry Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94:103–114, 2017.